

AI DOCUMENT INTELLIGENCE

End-to-End Financial Document Extraction & Validation

UPLOAD	EXTRACT	VALIDATE	PERSIST	DEPLOY
PDF / PNG / JPG	OCR + Gemini	Deterministic rules	SQLite + JSON	FastAPI + Render

Technical Case Study • AI Engineer Internship

FastAPI • PyMuPDF • Tesseract OCR • Google Gemini • SQLite • HTML/CSS/JavaScript • Docker • Render

01 Executive Summary

The challenge: turn messy financial documents into trustworthy, reviewable structured data.

4	3	2	1
supported document types	max pages per document	extraction paths: native + OCR	deterministic validation layer

- The user selects the document type and uploads a PDF, PNG, or JPG.
- Validation happens before extraction; corrupt, unsupported, oversized, or over-limit files are rejected.
- Native PDF text is extracted when available; scanned/image content goes through Tesseract OCR.
- Gemini converts page-aware text into structured fields and tables while being instructed not to invent missing values.
- Python formulas independently validate financial relationships and produce PASS / FAIL / NOT_APPLICABLE outcomes.
- Results are persisted and exposed through the dashboard and REST API.

02 Problem → Solution

PROBLEM	SOLUTION
Financial information is buried in PDFs, tables, and scans.	A unified pipeline converts documents into structured, reviewable JSON.
OCR can recover text but cannot reliably understand financial fields.	Gemini performs semantic extraction after OCR/text extraction.
LLMs should not be trusted for arithmetic validation.	Deterministic Python rules independently verify financial equations.
Extracted results need traceability.	Evidence text and page numbers are supported in the extraction design.

03 System Architecture

USER	WEB UI	FASTAPI	AI / OCR PIPELINE	RESULT
Upload Select type	HTML CSS JavaScript	Validation REST API	PyMuPDF → Tesseract → Gemini → JSON → Financial Rules	Dashboard SQLite Swagger API

External AI: Google Gemini is used only for document understanding and structured extraction. **Core financial validation remains deterministic Python logic.**

Persistence: SQLite stores document metadata, extracted JSON, validation JSON, status, and timestamp.

04 Supported Financial Documents

TYPE	EXTRACT	VALIDATE
INVOICE	Invoice number/date, parties, currency, totals, line items	Subtotal – Discount + Tax = Total
BALANCE SHEET	Assets, liabilities, equity and visible statement values	Assets = Liabilities + Equity
PROFIT & LOSS	Revenue, COGS, gross profit, income, expenses, net profit	Gross Profit = Revenue – COGS; Net Profit = Income – Expenses
CASH FLOW	Operating, investing, financing, opening/closing cash	Net Cash = O + I + F; Closing = Opening + Net Cash

Important: Missing or unreadable values are represented as null rather than invented.

05 Processing Pipeline

1	UPLOAD	PDF / PNG / JPG
2	VALIDATE	Type • integrity • size • ≤ 3 pages
3	EXTRACT	Native PDF text or Tesseract OCR
4	UNDERSTAND	Gemini field + table extraction
5	STRUCTURE	JSON fields • tables • evidence • page
6	CHECK	Deterministic financial formulas
7	STORE	SQLite + API response
8	DISPLAY	Dashboard + Swagger / REST

- Validation is intentionally placed before expensive AI processing.
- Page-aware extraction helps preserve source context.
- Financial validation is independent of the LLM output generation step.

06 Financial Validation

The application treats arithmetic as a deterministic engineering responsibility rather than an LLM responsibility.

CASE	EXPECTED CALCULATION	ACTUAL	STATUS
Invoice	$100 - 10 + 9$	99	PASS
Balance Sheet	$400 + 600$	1,000	PASS
Profit & Loss	$1,000 - 400$	600	PASS
Cash Flow	$500 - 100 + 100$	500	PASS

- Tolerance: 0.01 for numeric comparisons.
- Each validation rule returns PASS, FAIL, or NOT_APPLICABLE when required inputs are unavailable.
- The overall validation result is persisted with the extracted document.

07 AI Extraction Design

Gemini receives the extracted, page-aware text rather than being asked to directly inspect an uncontrolled file workflow.

- **Grounded extraction:** use only information present in the supplied document text.
- **No hallucination:** if a value is missing or unreadable, return null.
- **Structured output:** fields and tables are returned as JSON.
- **Traceability:** evidence/source text and page number can accompany values.
- **Separation of concerns:** Gemini extracts; Python validates.

Example structured result

```
{
  "document_type": "invoice",
  "fields": {
    "invoice_number": "INV-001",
    "currency": "INR",
    "subtotal": 10000,
    "discount": 500,
    "tax": 1710,
    "total": 11210
  },
  "financial_validation": {"overall_status": "PASS"}
}
```

08 API, Dashboard & Persistence

METH OD	ENDPOINT	PURPOSE
POST	/api/v1/documents/process	Upload and process a document
GET	/api/v1/documents/	List processed documents
GET	/api/v1/documents/{id}	Retrieve extracted + validation JSON
GET	/api/v1/documents/search/by-name	Search stored documents
GET	/api/v1/health	Health check
GET	/docs	Swagger / OpenAPI documentation

- SQLite persists document metadata and JSON results.
- The frontend provides upload, processing status, extracted fields, tables, and validation results.
- Swagger provides an immediately testable API contract.

09 Deployment & Security Basics

LOCAL	PRODUCTION
Python 3.11 virtual environment FastAPI + Uvicorn Tesseract OCR SQLite	Docker container Render Web Service Environment variables Health check: /api/v1/health

- Secrets are stored in environment variables and the .env file is excluded from Git.
- Docker installs the Tesseract system dependency required by the OCR path.
- Live service: <https://ai-document-intelligence-bok5.onrender.com/>

10 Testing & Quality

EXTRACTION	VALIDATION	INTEGRATION	DEPLOYMENT
4 document types	PASS + FAIL cases	API + persistence	Live health check

- Automated tests cover invoice, balance sheet, profit & loss, and cash flow validation.
- Extraction tests cover representative fields for all four supported types.
- The test suite includes financial failure scenarios.
- The deployed service was health-checked through the production endpoint.
- The repository contains sample JSON outputs and deployment configuration.

11 Sample Outputs

FILE	TYPE	RESULT	EXAMPLE
sample.json	Invoice	PASS	Total = 11,210 INR
balance_sheet_sample.json	Balance Sheet	PASS	Assets = 1,000,000
profit_loss_sample.json	Profit & Loss	PASS	Net Profit = 300,000
cash_flow_sample.json	Cash Flow	PASS	Closing Cash = 1,500,000

These samples demonstrate the intended structured JSON contract and successful deterministic validation.

12 Limitations & Production Roadmap

CURRENT MVP	NEXT PRODUCTION STEP
SQLite persistence	PostgreSQL + object storage
Synchronous processing	Background queue / workers
User-selected document type	Automatic document classification
Basic dashboard	Authentication, roles, audit trail
OCR depends on scan quality	Image preprocessing + confidence thresholds
Single AI provider	Provider abstraction + fallback strategy

13 Conclusion

From document upload to validated financial insight

This project demonstrates a complete AI document-intelligence workflow rather than an isolated AI model: validated ingestion, native/OCR text extraction, grounded Gemini extraction, structured JSON, deterministic financial validation, persistence, REST APIs, dashboard presentation, automated tests, Docker packaging, and public deployment.

AI / Tool Declaration

Google Gemini: semantic field/table extraction. PyMuPDF + Tesseract: text/OCR. Python: deterministic financial validation. FastAPI: backend/API. SQLite: persistence. HTML/CSS/JavaScript: frontend. Docker + Render: deployment.

Live application: <https://ai-document-intelligence-bok5.onrender.com/>